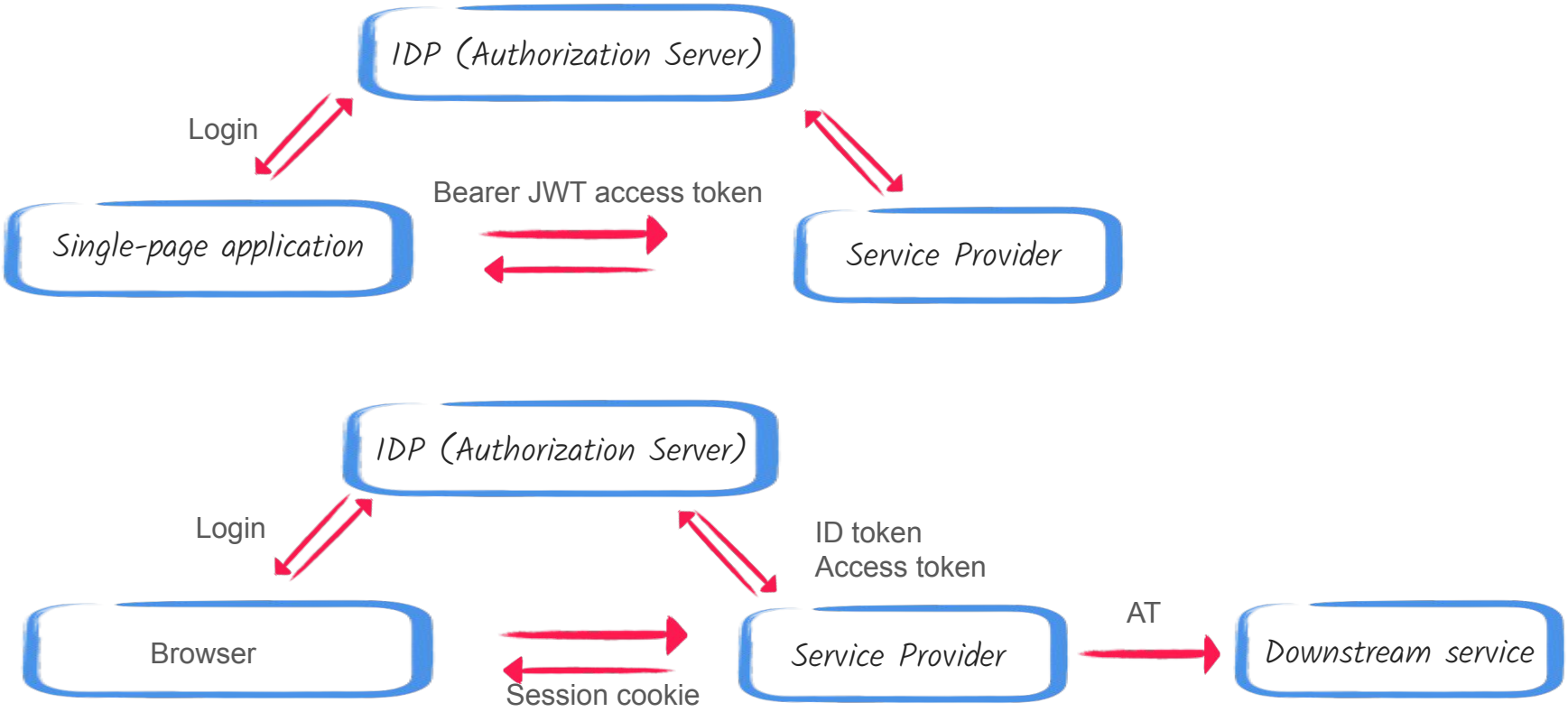
Introduction to Verifiable Credentials

Sergey Beryozkin
IBM, Quarkus Security
@sberyozkin
sbiarozk@ibm.com

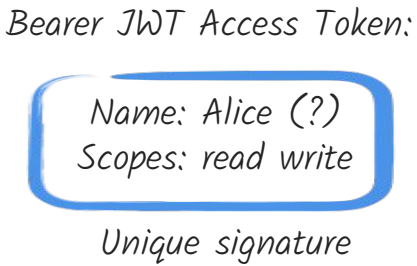
Image: generated by Google Nano Banana

Traditional Federated (IDP) Architecture



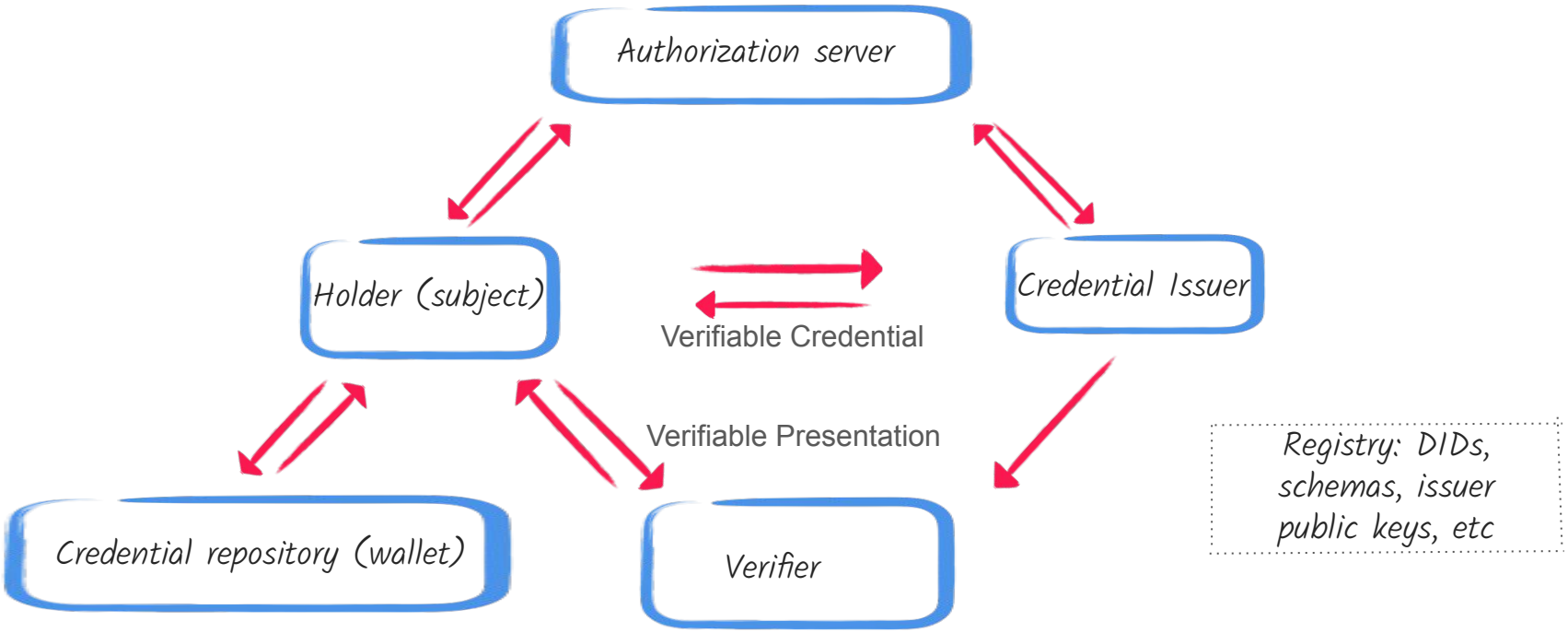
Traditional IDP architecture and token properties

- OpenId Connect (OIDC) Provider knows every SP the user visits, since SP acts as an OpenId Connect client that must be registered with IDP.
- JWT token is typically immediately sent from IDP to SP
- All JWT token claims are visible to SPs: full disclosure scheme
- The same JWT token signature is visible to all SPs that this token is sent to



All Service Providers the accept these tokens see the same claims, often immediately after the token issuance

Issuer-Holder-Verifier Architecture



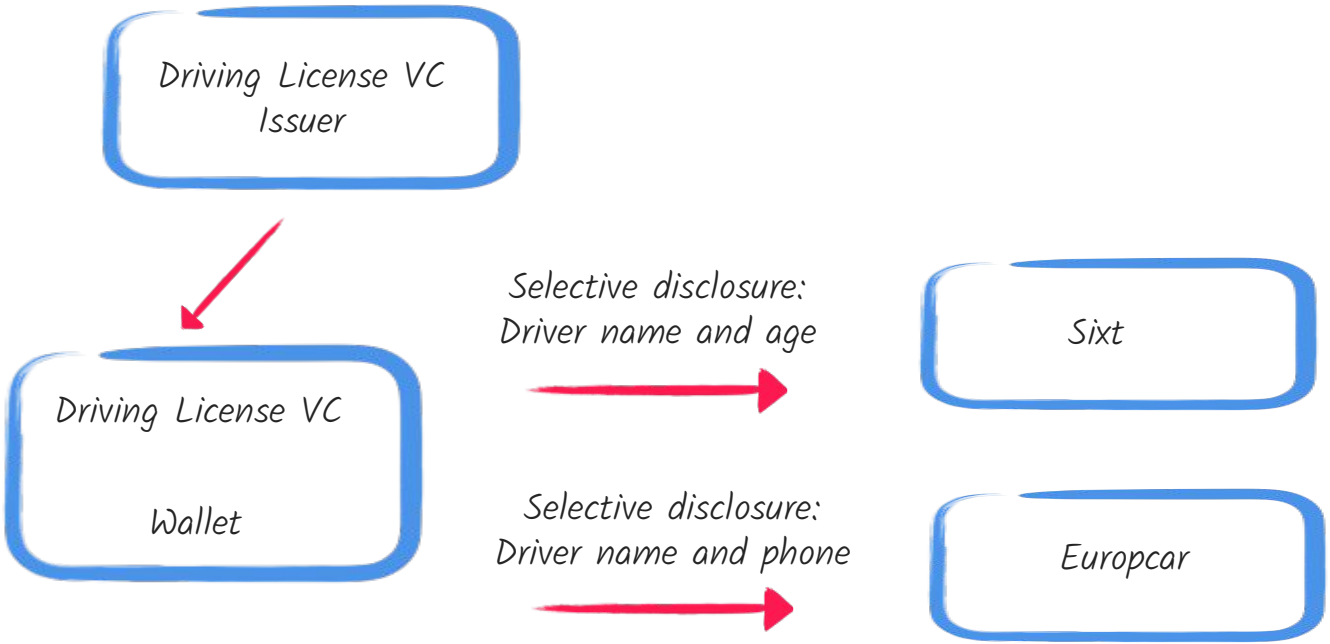
Issuer-Holder-Verifier architecture properties

- VCs often act as digital, cryptographilcally secured analogs of physical credentials such as a driving license
- Focus on privacy, by applying Selective Disclosures and Zero Knowledge Proofs to presentations
- Issuer-Holder-Verifier Model decouples the issuance of a JWT from its presentation

Architecture is decentralized, attempts to align with how we usually work with physical documents in the real life

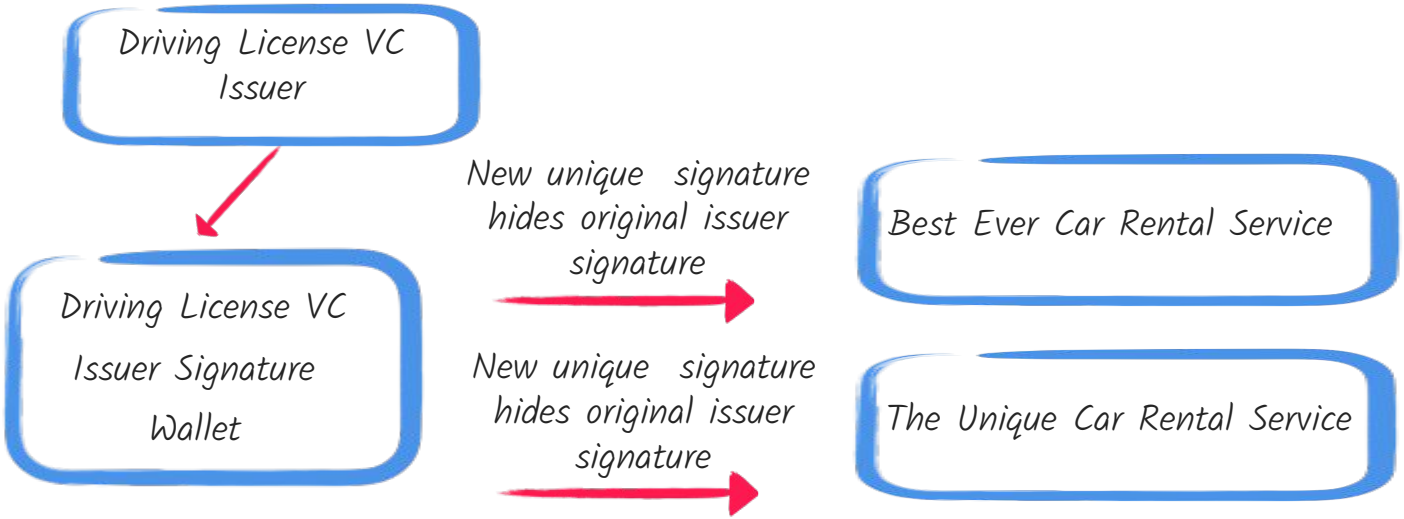
Selective Disclosures

- SD-JWT is currently the most popular mechanism for selective disclosures



Zero Knowledge Proof

- **Zero Knowledge:** I'm older than 20 years old without disclosing an actual age
- **Unlinkable Proofs**: When proofs generated by the scheme are zero-knowledge, proofs of knowledge of the signature, meaning a verifying party in receipt of a proof is unable to determine which signature was used to generate the proof, removing a common source of correlation.
    - Holder does not require access to the issuer's private key !
- Good summary of many interesting options: https://ageverification.dev/av-doc-technical-specification/docs/annexes/annex-B/annex-B-zkp/



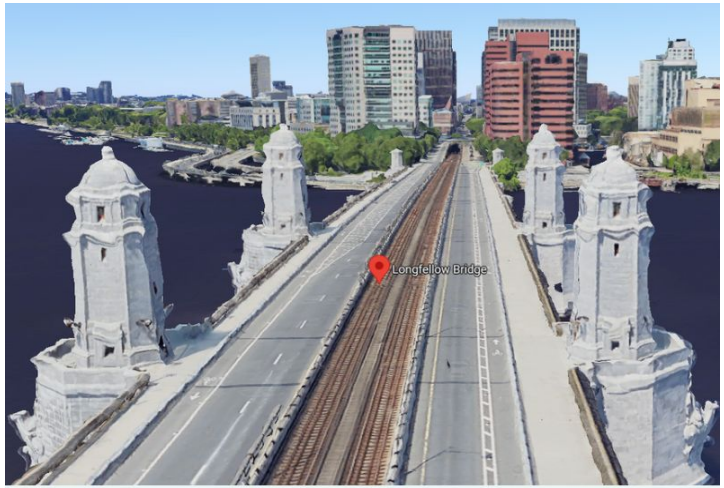
"Best Ever" and "The Unique" car rental services can not collude to determine that the same customer who presented Zero Knowledge VC visited their websites

Zero Knowledge Proof: BBS Signatures

- BBS Signatures is named for its academic originators Dan **B**oneh, Xavier **B**oyen, and Hovav **S**hacham. A provably secure version, called BBS+ was devised later, see https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/, https://github.com/mattrglobal/bbs-signatures, etc
- The primary pairing-friendly elliptic curve used for the BBS and BBS+ signature schemes is **BLS12-381**
- **BLS12-381** is based on the discrete logarithm problem, therefore it is not Post Quantum Cryptography resistant, however, the privacy and hiding properties of BBS proofs are resilient
- Proof of ownership is naturally supported at the BBS Signature level (in contrast, for example, to OAuth2 DPoP)
- BBS Signature is among the supported JSON Web Proof algorithms

Zero Knowledge Proof: zk-SNARK

- Google: Safeguarding cryptocurrency by disclosing quantum vulnerabilities responsibly: https://research.google/blog/safeguarding-cryptocurrency-by-disclosing-quantum-vulnerabilities-responsibly/
- “In this Appendix, we provide a **Zero-Knowledge (ZK) proof** to substantiate the resource estimates for breaking the 256-bit Elliptic Curve Discrete Log Problem (ECDLP) described in the main text, **without disclosing our improved logical circuits**. We use the SP1 Zero-Knowledge Virtual Machine (zkVM) [83] to generate a Groth16 **Zero Knowledge Succinct Non-interactive ARgument of Knowledge** (**SNARK**) [233] proof of the fact that we possess a quantum circuit that correctly implements elliptic curve point addition on the secp256k1 curve [84, 85] and satisfies certain resource constraints.
- zk- SNARK is used by Google Longfellow-zk and can be applicable to the Verifiable Credentials space



The Longfellow bridge in Boston near the Google HQ, as seen by Google maps.

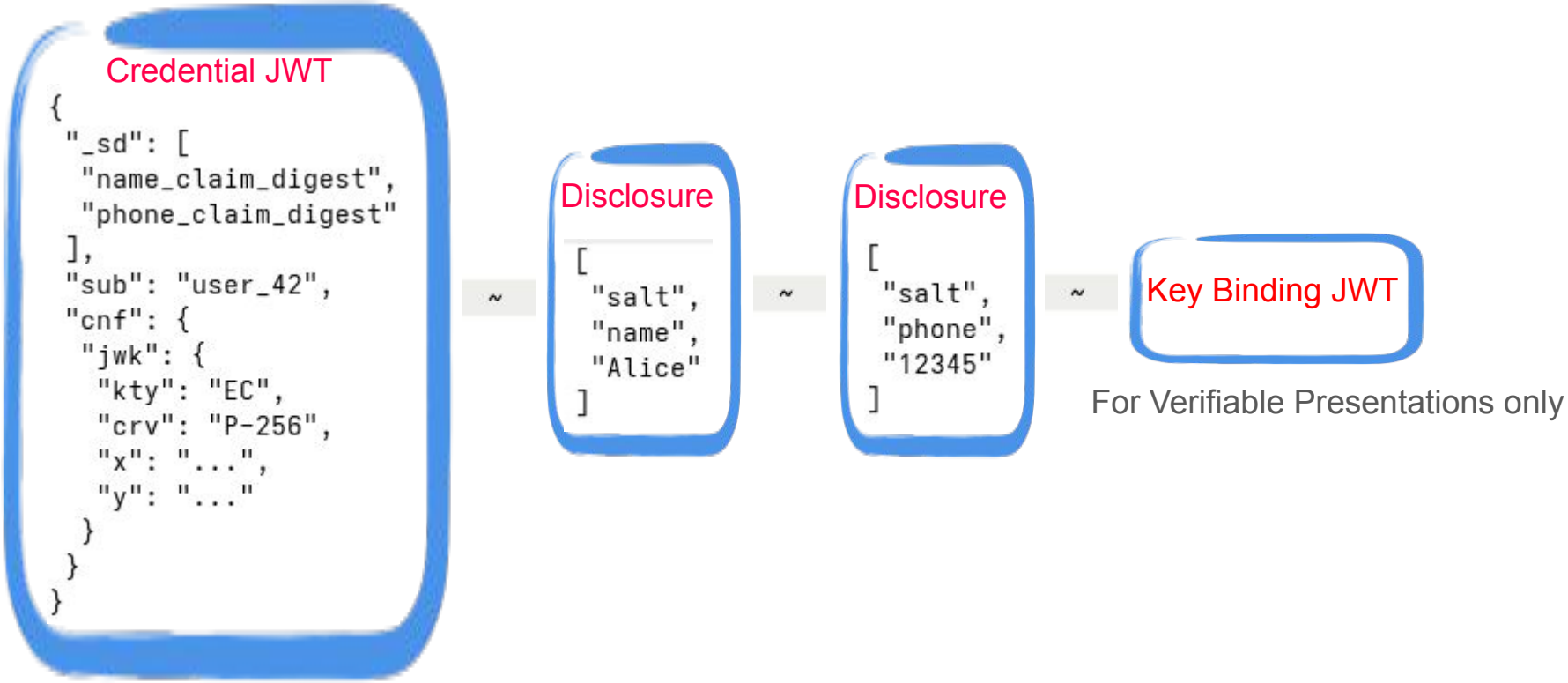
https://news.dyne.org/longfellow-zero-knowledge-google-zk/

Zero Knowledge Proof: Longfellow-zk vs SD-JWT

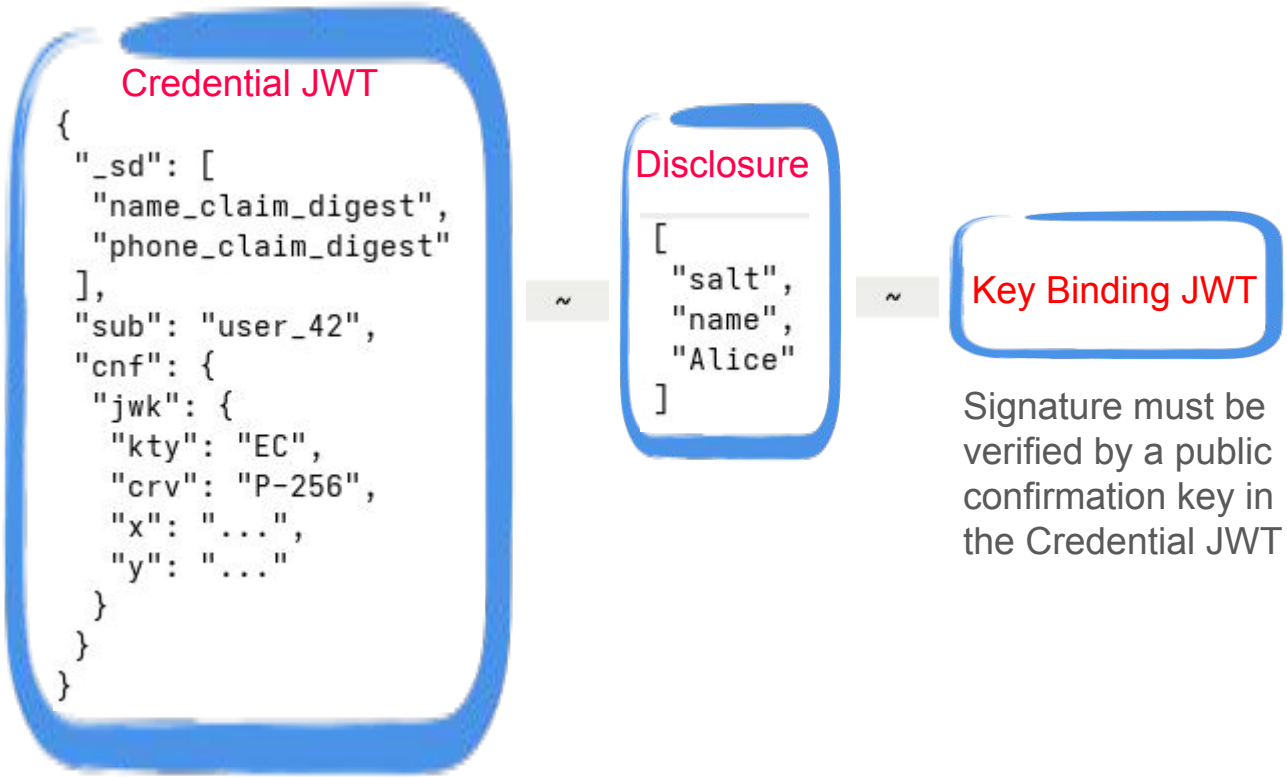
- Improved Privacy (True Zero-Knowledge)
  - Longfellow-ZK: Uses zkSNARKs to prove a statement about a credential (e.g., "I am over 18") without revealing the actual birthdate or any other information in the credential.
  - SD-JWT: Is a "selective disclosure" mechanism, not a zero-knowledge system. It reveals the exact value of a claim (e.g., "birthdate: 1990-01-01") to the verifier, merely hiding other non-disclosed claims.
- Issuer Infrastructure:
  - Longfellow-ZK: Works with existing, signed ECDSA (P-256) credentials or mDOCs. It does not require re-issuing credentials, making it immediately compatible with current infrastructure.
  - SD-JWT: Requires specific, new issuance of tokens with hidden claims (_sd_alg) and managed disclosures, requiring issuers to update their systems.
- But what about Post Quantum Cryptography ?
  - Longfellow-ZK: Works with existing, signed ECDSA, the question of supporting ML-DSA is mute right now
  - SD-JWT will naturally get PQC-resistant ML-DSA signatures supported once a JOSE standard for signing JWT tokens with ML-DSA is finalized (in fact, RFC-9964 got published recently)

SD-JWT

- SD-JWT is a composite structure, consisting of JWT plus optional Disclosures and a Key Binding
- SD-JWT VC uses SD-JWT as basis for representing Verifiable Credentials with JSON payloads
- https://datatracker.ietf.org/doc/html/rfc9901, https://datatracker.ietf.org/doc/draft-ietf-oauth-sd-jwt-vc/



SD-JWT Verifiable Presentation

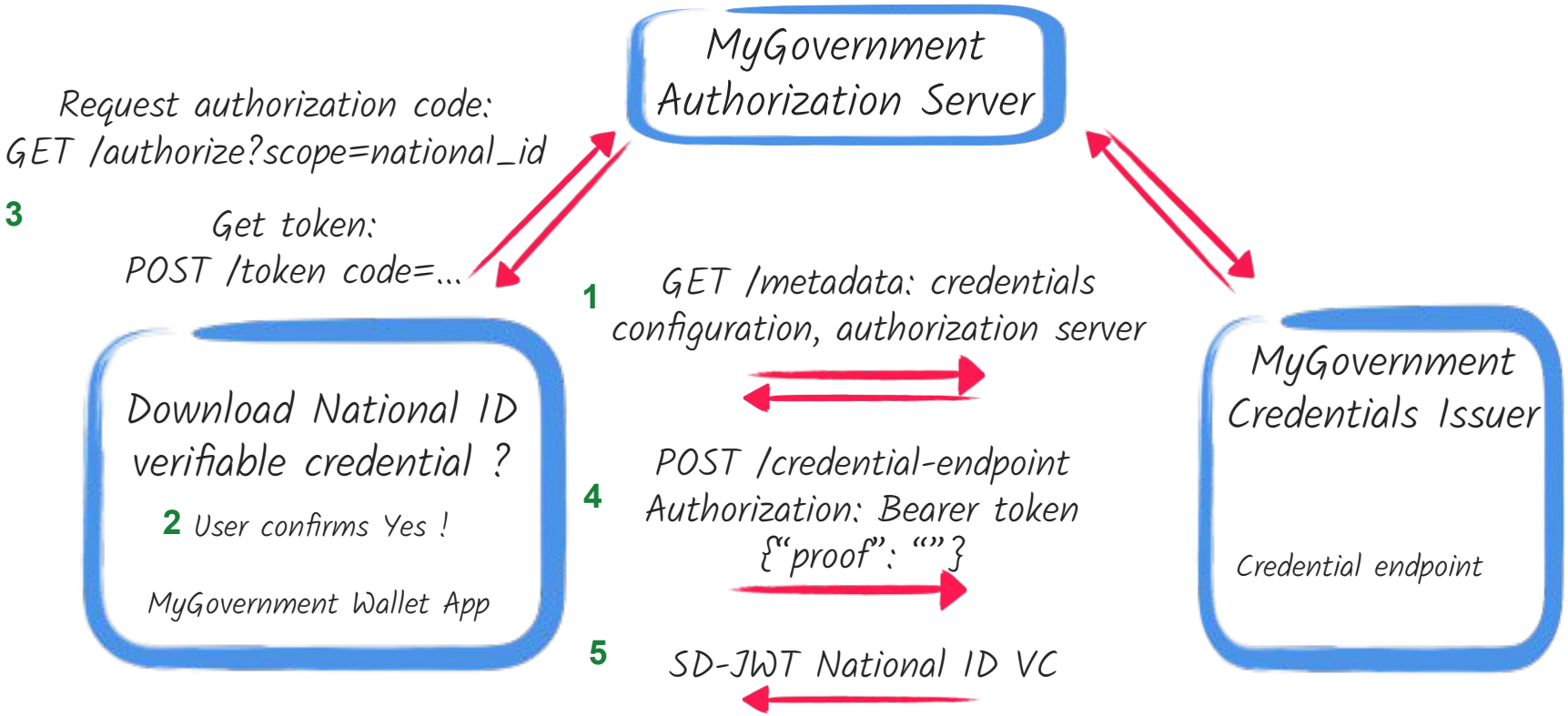


Signature must be verified by a public confirmation key in the Credential JWT

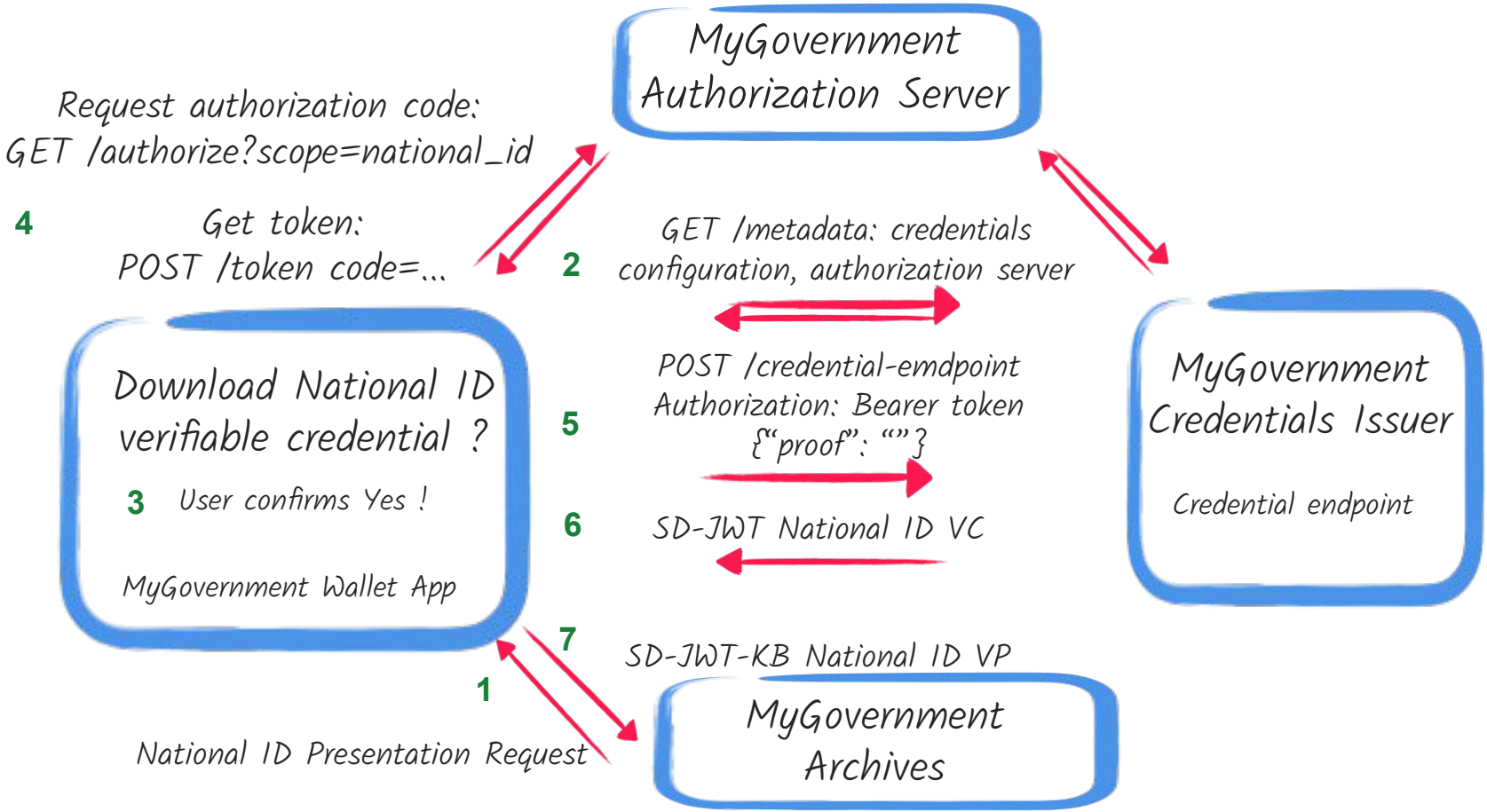
OpenID for Verifiable Credential Issuance (OID4VCI)

- OID4VCI defines an OAuth-protected API for the issuance of Verifiable Credentials
- Credential Issuer metadata
- Credential endpoint
- Wallet-initiated flow
  - Wallet offers users an option to download well known VCs
  - Wallet acquires VC following a user consent to a VP request from the verifier
- Issuer initiated flow:
  - User browser a university website, selects a required credential
  - Credential offer endpoint
- Authorization code flow
- New grant type: Pre-authorized grant
- Deferred credential issuance

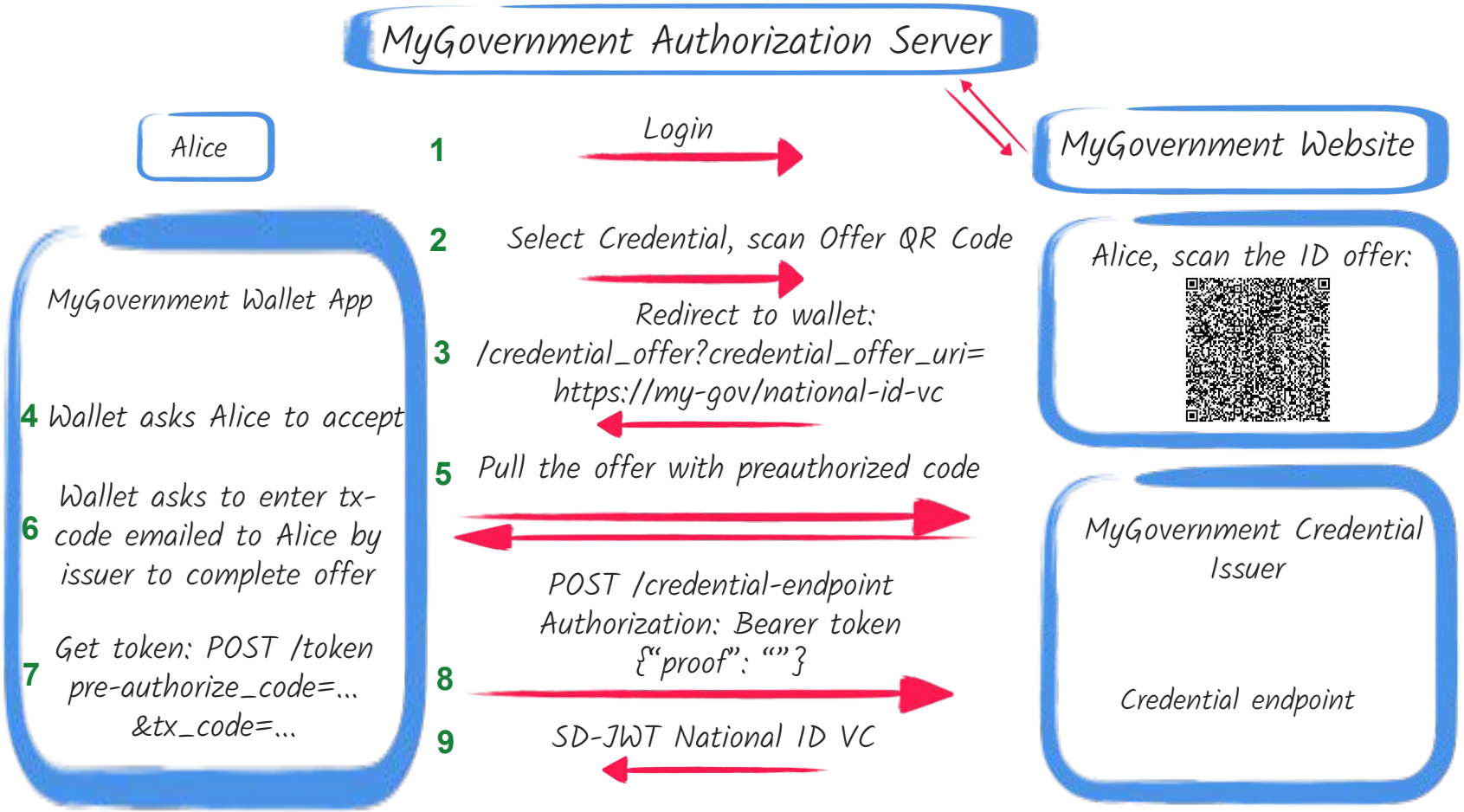
OID4VCI: Wallet Initiated Flow: Well Known Issuer



OID4VCI: Wallet Initiated Flow: Verifier Presentation Request



OID4VC: Issuer Initiated Flow: Pre-authorized code grant



OID4VCI: Credential Endpoint: Proof of Possession bound to VC

Wallet sends a proof of knowledge of a private key to the issuer's credential endpoint:

POST /credential-endpoint
Authorization: Bearer token
{"proof": ""}


```
{
  "jwt": [
    "eyJ0eXAiOiJvcGVuaWQ0dmNpLXByb29mK2p3dCIsImFsZyI6IkVTMjU2Iiwiand
    rIjp7Imt0eSI6IkVDIiwiY3J2IjoiUC0yNTYiLCJ4IjoiblVXQW9BdjNYWml0aDh
    FN2kxOU9kYXhPTFlGT3dNLVoyRXVNMDJUaXJUNCIsInkiOiJIc2tIVThCalVpMVU
    5WHFpN1N3bWo4Z3dBS18weGtjRGpFV183MVNvc0VZIn19.eyJhdWQiOiJodHRwcz
    ovL2NyZWRlbnRpYWwtaXNzdWVyLmV4YW1wbGUuY29tIiwiaWF0IjoxNzAxOTYwND
    Q0LCJub25jZSI6IkxhclJHU2JtVVBZdFJZTzZCUTR5bjgifQ.-a3EDsxClUB4O3L
    eDD5DVGEnNMT01FCQW4P6-2-BNBqc_Zxf0Qw4CWayLEpqkAomlkLb9zioZoipdP-
    jvh1WlA"
  ]
}
```


where the decoded JWT looks like this:

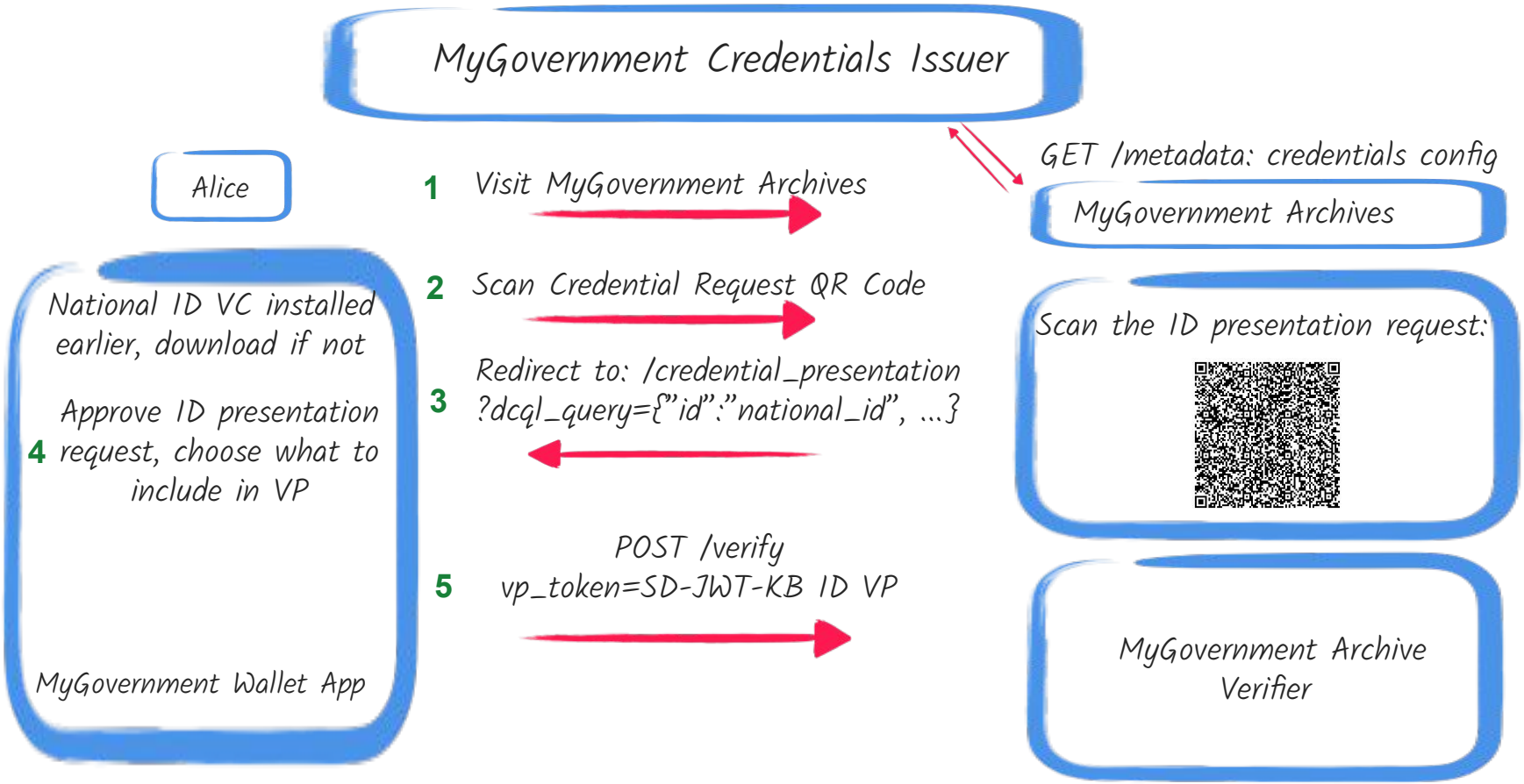

```
{
  "typ": "openid4vci-proof+jwt",
  "alg": "ES256",
  "jwk": {
    "kty": "EC",
    "crv": "P-256",
    "x": "nUWAoAv3XZith8E7i19OdaxOLYFOwM-Z2EuM02TirT4",
    "y": "HskHU8BjUi1U9Xqi7Swmj8gwAK_0xkcDjEW_71SosEY"
  }
}.{
  "aud": "https://credential-issuer.example.com",
  "iat": 1701960444,
  "nonce": "LarRGSbmUPYtRYO6BQ4yn8"
}
```


This public key in JWK format will be included in SD-JWT-VC signed by the issuer, holder will use its private key to create a key binding in SD-JWT-KB for verifier to verify a proof of ownership by the holder

OpenID for Verifiable Presentations (OID4VP)

- OID4VP defines a “mechanism to request and present Credentials”
- New response_mode=direct_post, new response_type=vp_token
  - OAuth2 authorization code already supports response modes such as `query` and `form_post`, while a typical response type is `code`.
- Integration with Digital Credential APIs
- Use case:
  - User is looking for a job and finds a website that accepts VPs
  - User is redirected to the wallet and a wallet initiated VC acquisition flow starts
  - Wallet completes a `direct_post` response mode by POSTing a `vp_token` to the verifier

OID4VP: response_mode=direct_post & response_type=vp_token



Do I really care about my driving license containing my name and age ?

- You may want to ask: What is a big deal about Verifiable Credentials, all the privacy considerations, unlinkability ? I don’t mind if my driving license contains my name and even address ?
- Plausible answer: once Verifiable Credentials go mainstream, and you have a choice between presenting a zero-knowledge proof document and a traditional document with all the details like your age, address and phone, you will choose the former option
- There will be a next to unlimited number of cases where users will prefer to not disclose some information during the credential presentation

Imagine: Claim your University of Coimbra Digital Degree Credential

UNIVERSIDADE D COIMBRA

1 2 90

ALUMNI

Community

Request digital degree credential

UC Alumni Card

Alumni Associations

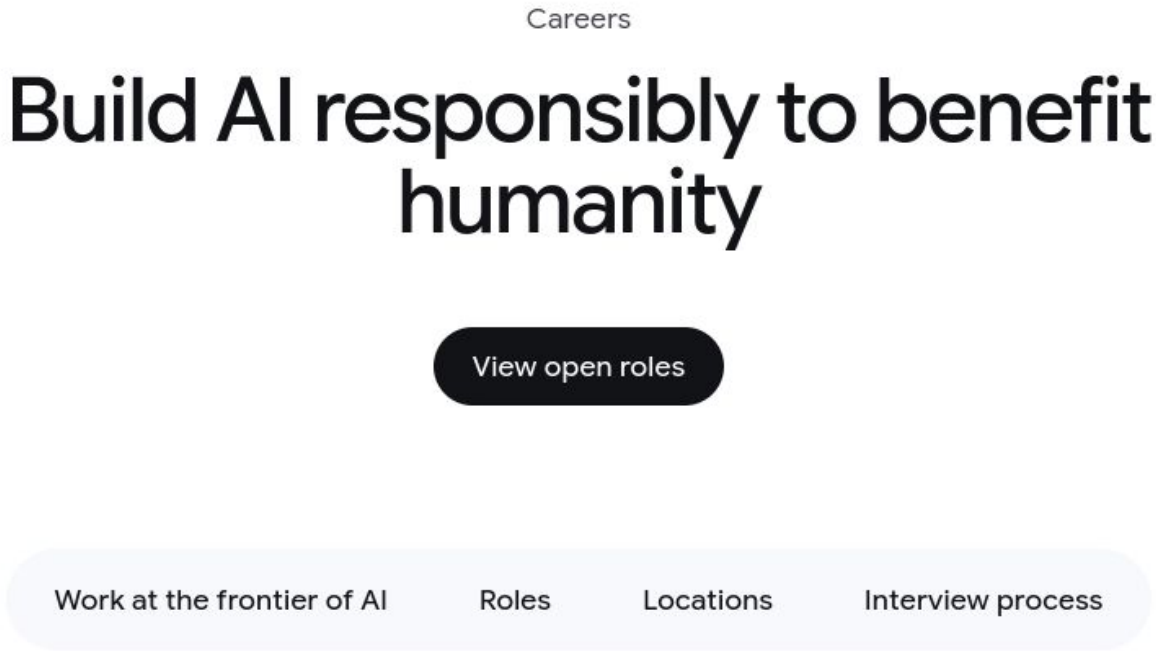
Alumni Associations

University of Coimbra

Find the UC Alumni association near you.

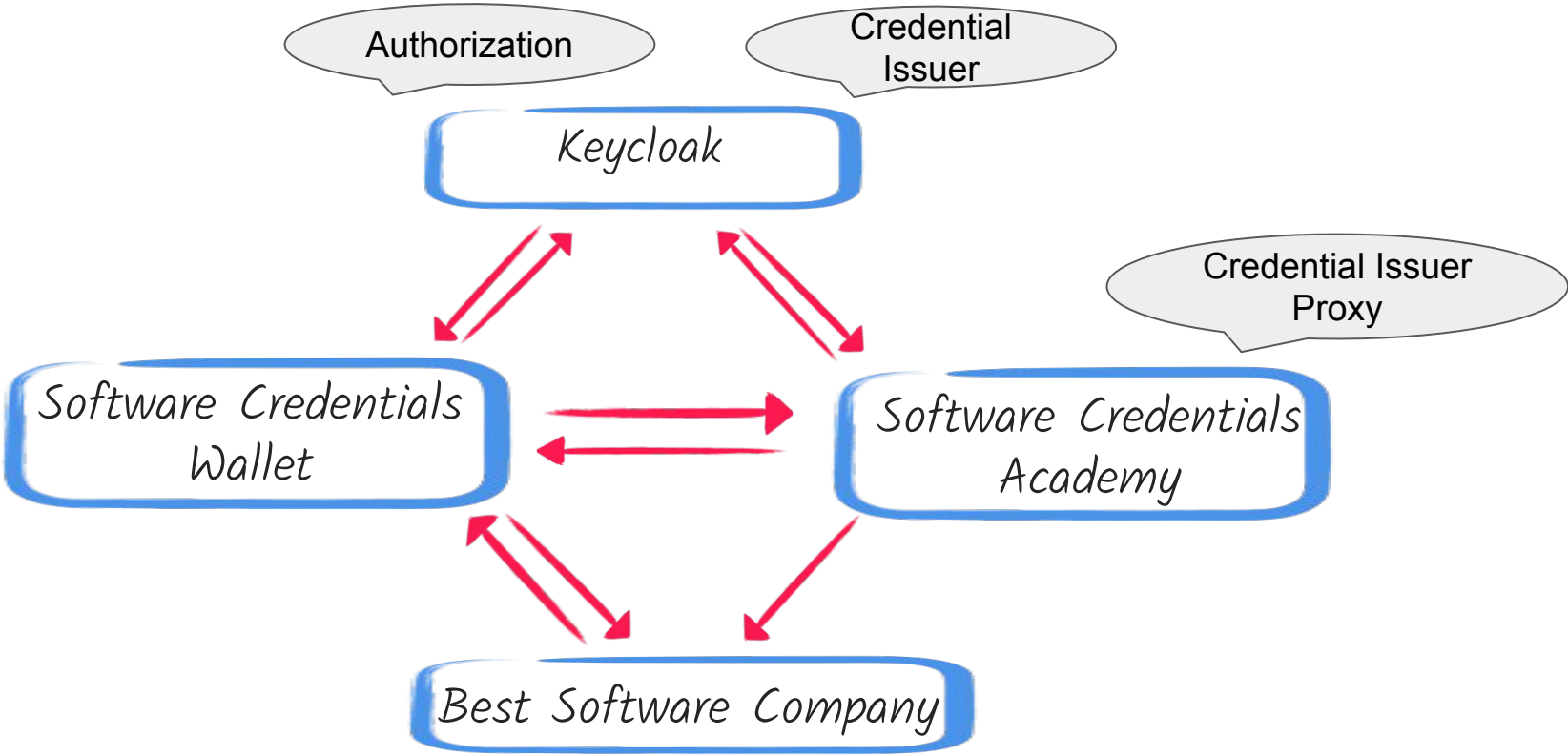
UC around the world

Imagine: Present Coimbra AI course digital credential to apply for a job at Google DeepMind



Present AI course digital credential

Demo: Verifiable Software Credentials



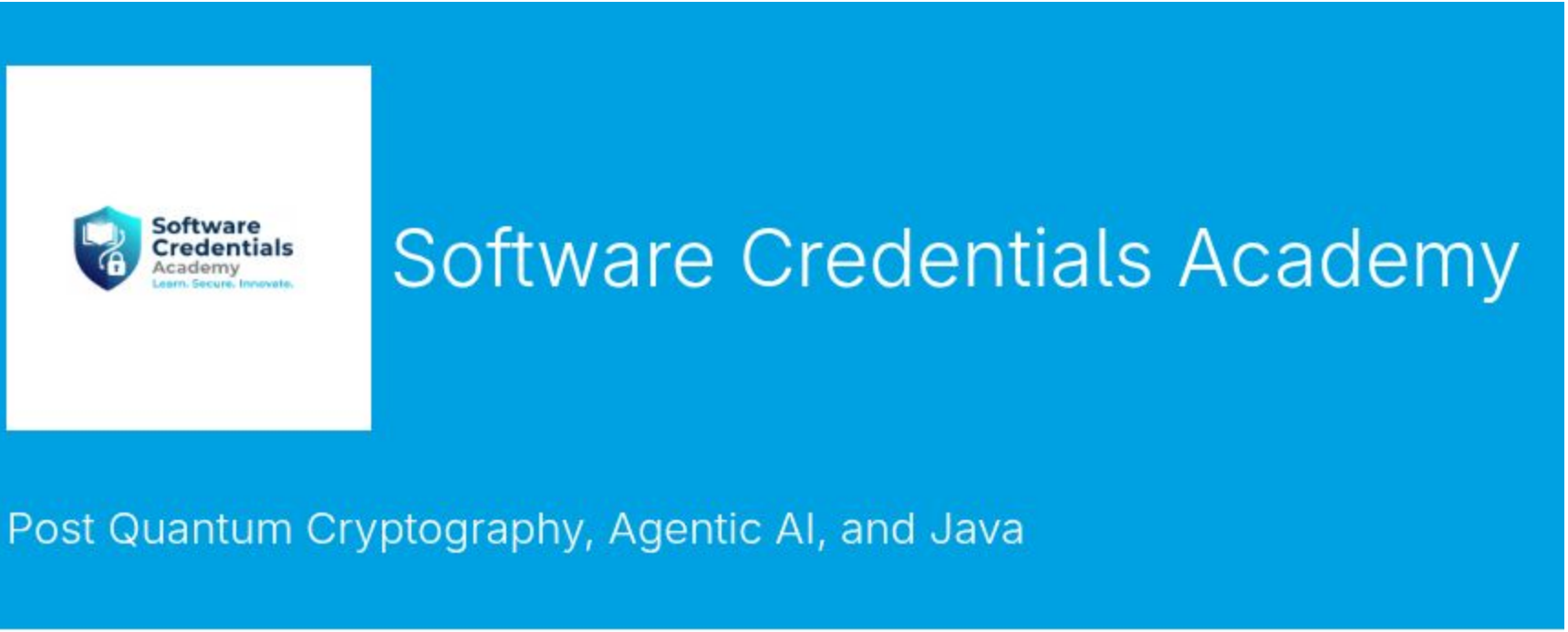
WIP:
https://github.com/quarkiverse/quarkus-verifiable-credentials : Turn your REST endpoint into Verifier
https://github.com/sberyozkin/quarkus-quickstarts/tree/oidc-vc/security-verifiable-credentials-quickstart: Demo

Demo login:
user name - “alice”, password: “alice”

Thank You !

- Session slides: https://github.com/sberyozkin/quarkus-quickstarts/tree/oidc-vc/security-verifiable-credentials-quickstart/introduction_to_vc.pdf
- Software Credentials Demo (Wallet emulator, Issuer, Verifier): https://github.com/sberyozkin/quarkus-quickstarts/tree/oidc-vc/security-verifiable-credentials-quickstart
- Please watch https://github.com/quarkiverse/quarkus-verifiable-credentials, propose ideas, this WIP Quarkus extension will help turn REST endpoints into Verifiable Presentation Verifiers capable of accepting SD-JWT VC presentations to start with, eventually - also zero-knowledge proofs
- Would you like to follow up with some questions ? You can always find me at https://quarkusio.zulipchat.com/, https://www.linkedin.com/in/sberyozkin/

Demo backup in case of WiFi issues



Login to enroll or claim a digital course credential

Software Credentials Academy

Post Quantum Cryptography, Agentic AI, and Java

Alice, request a software credential:

Post Quantum Cryptography PhD

Agentic AI Security Course

Java Coder Champion

Post Quantum Cryptography PhD offer

Web Link QR Code:

[Open in Wallet]

[Software Credentials Academy Home]

Mailpit · Mailpit UI

Mailpit

Search mailbox

Inbox 1

Mark all read

Delete all

transaction-codes@software-credentials-aca... **Transaction code to complete PostQuantumCryptographyPhD offer**
To: alice@emailprovider.com 3942

Post Quantum Cryptography PhD credential offer from Software Credentials Academy

Credential:

Post Quantum Cryptography PhD

Enter transaction code: 3942

Accept

Wallet Dashboard

Software Credentials Wallet

Alice, here are your software credentials:

Post Quantum Cryptography PhD

Demo: Software Credentials Verifier

Would you like to join our team ? We accept the following credentials:

Post Quantum Cryptography PhD

Agentic AI Security Course

Java Coder Champion

Agentic AI Security Course credential presentation request from Best Software Company

Credential:

Agentic AI Security Course

Download requested credential

Wallet Dashboard

Agentic AI Security Course presentation

Claims that can be disclosed:

- [x] ai_course_provider : Software Credentials Provider
- [x] given_name : Alice
- [ ] family_name : Brown
- [ ] username : alice

Present to Best Software Company

Wallet Dashboard

Agentic AI Security Course presentation

Disclosed claim values:

ai_course_provider	Software Credentials Provider
given_name	Alice

Best Software Company Home